



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
IIC2233 — PROGRAMACIÓN AVANZADA 2026-1

Interrogación 1

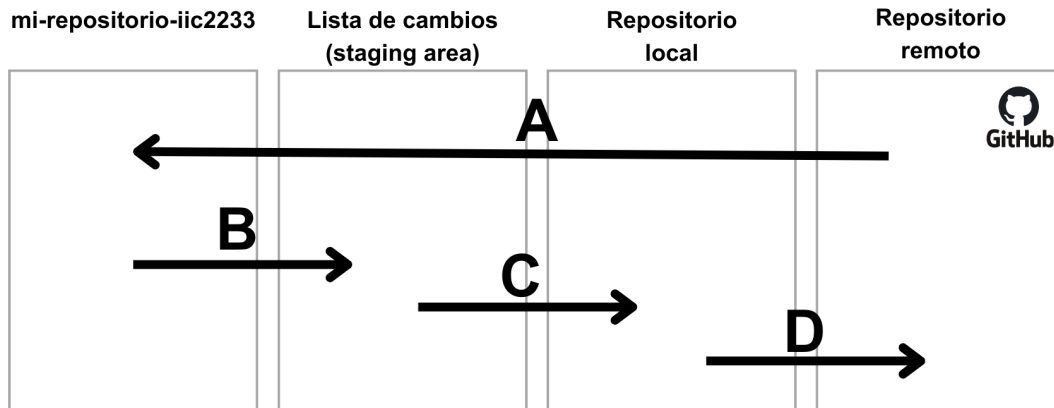
15 de Abril 2026

Instrucciones

- La evaluación consta de dos partes: 5 preguntas de selección múltiple y 4 preguntas de desarrollo.
- La interrogación consta de una sección de alternativas y una sección de desarrollo. La sección de desarrollo está compuesta por 4 preguntas. Tanto la sección de alternativas como cada una de las preguntas de desarrollo valen un 20 % de la nota final.
- Recibirás una hoja de respuestas que deberás rellenar con tus datos y las respuestas correspondientes a cada alternativa o pregunta de desarrollo. Sólo se corregirá la hoja de respuestas. **Ten mucha precaución de anotar correctamente tus datos.**
- Cada pregunta de selección múltiple tiene únicamente una alternativa correcta. Marcar dos o más alternativas invalidará la pregunta, considerándose incorrecta. Todas las preguntas tienen igual puntaje y no existe penalización por respuestas incorrectas. Se permite justificar la alternativa seleccionada en la hoja de respuesta; esta justificación será considerada en caso de corrección de la pregunta.
- Debes completar con tus datos personales en cada hoja utilizada. Luego debes entregar de vuelta **todas** las hojas de respuesta (tanto para alternativas como de desarrollo) de la prueba, aunque estas se encuentren en blanco.
- Solo podrás retirarte de la sala una vez hayan transcurrido 60 minutos desde que inició de la evaluación.
- Durante la evaluación se realizarán 2 rondas de preguntas. Estas preguntas únicamente serán respondidas por los profesores del ramo.
- En caso de que sea necesario, podrás solicitar hojas blancas para apoyar al desarrollo de la prueba. Para esto levanta la mano y espera que un ayudante se acerque a tu puesto.

Selección múltiple [20 % nota final]

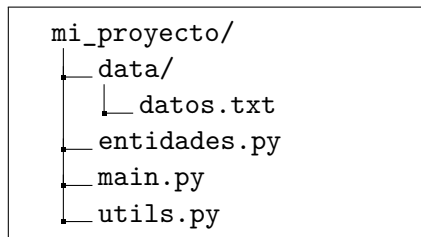
1. A partir del siguiente diagrama de flujo de trabajo en **Git**, y considerando lo visto en clases, en el que cada flecha (A, B, C y D) representa una operación entre los distintos espacios de trabajo.



¿Cuál de las siguientes alternativas asigna **correctamente** cada letra con su operación correspondiente? Suponga que el repositorio ya existe en **GitHub**, que no hay problemas de conexión y que todas las operaciones se ejecutan correctamente.

- A) A: git pull
B: git push
C: git add README.md
D: git commit -m "Mucho éxito"
- B) A: git pull
B: git commit -m "Mucho éxito"
C: git add README.md
D: git push
- C) A: git push
B: git add README.md
C: git commit -m "Mucho éxito"
D: git pull
- D) A: git pull
B: git add README.md
C: git commit -m "Mucho éxito"
D: git push
- E) A: git pull
B: git add README.md
C: git push
D: git commit -m "Mucho éxito"

2. ¿Por qué se **recomienda** utilizar un sistema de control de versiones como **Git** durante el desarrollo de código?
- A) Permite mantener un respaldo del trabajo realizado.
 - B) Permite recuperar versiones anteriores del proyecto.
 - C) Facilita la colaboración entre múltiples desarrolladores.
 - D) Permite llevar un registro ordenado de los cambios realizados.
 - E) Todas las anteriores.
3. Considera la siguiente estructura de archivos de un proyecto y el contenido del archivo `utils.py`:



```
from math import *

def leer_archivo():
    path = "./data/datos.txt"
    with open(path, "r") as archivo:
        contenido = archivo.read()
    return contenido

def calcular_area_total(*radios):
    total = 0
    for r in radios:
        total += pi * r**2
    return round(total, 2)
```

Considere que el programa se ejecuta desde la carpeta `mi_proyecto` y que `main.py` funciona correctamente.

¿Cuál(es) de las siguientes afirmaciones es(son) **correcta(s)**?

- I. La organización del código en múltiples archivos favorece la modularización del proyecto.
 - II. El uso de imports en el archivo `utils.py` sigue buenas prácticas, ya que mejora la legibilidad y mantenibilidad del código.
 - III. La forma en que se define el `path` en `leer_archivo` garantiza la portabilidad del código en distintos entornos.
 - IV. El uso de `*radios` permite que la función reciba una cantidad variable de argumentos.
- A) I y III
 - B) I y IV
 - C) II y III
 - D) I, III y IV
 - E) II, III y IV

4. Un/a estudiante desarrolla el siguiente código en el archivo `main.py`:

```
1 import utils
2
3 lista_Estudiantes = []
4
5 def ProcesarData():
6     ruta = "C:/Users/estudiante/Desktop/T1/data/alumnos.txt"
7     with open(ruta, "r") as x:
8         for linea in x:
9             nombre, promedio = linea.strip().split(",")
10             lista_Estudiantes.append((nombre,float(promedio)))
11
12     utils(lista_Estudiantes)
13
14 ProcesarData()
```

Con respecto al código, ¿cuál o cuáles de las siguientes aseveraciones son **verdaderas**?

- I. La variable `ruta` almacena una ruta relativa.
- II. El código cumple las recomendaciones de estilo **PEP8**.
- III. Al ejecutar el archivo `main.py`, el programa no presenta errores.

- A) Solo I
- B) Solo III
- C) I y II
- D) I y III
- E) Ninguna es verdadera.

5. Considere que se está trabajando en un computador utilizando la terminal para ejecutar comandos. Tenga en cuenta lo siguiente:

- El comando “`mv origen destino`” permite mover archivos o carpetas de un lugar (**origen**) a otro (**destino**).
- El comando “`python3`” funciona correctamente en el computador.
- La carpeta “`test_publicos`” contiene tests válidos.

En este contexto, se ejecutan los siguientes comandos en la terminal, donde el símbolo “\$” indica el punto en que el usuario ingresa un comando:

```
$ ls
  repositorio
$ cd repositorio/proyecto/T0
$ ls
  test_publicos      main.py
$ mkdir ../../T1
$ mv main.py ../../T1
$ mv test_publicos ../../T1
$ python3 -m unittest discover test_publicos -v -b
```

Con respecto a esta situación, ¿cuál o cuáles de las siguientes aseveraciones son **correctas**?

- I. El usuario se encuentra en la carpeta “T1”.
- II. La carpeta “T1” está contenida en la carpeta “proyecto”.
- III. El comando “`python3 -m unittest discover test_publicos -v -b`” se ejecuta correctamente.

- A) Solo I
- B) Solo II
- C) Solo III
- D) Solo II y III
- E) Ninguna aseveración es correcta.

Preguntas Desarrollo

Pregunta 1. [20 % nota final]

En los siguiente segmentos de código se implementan clases que utilizan listas. En cada ejemplo proponga estructuras de datos o una combinación de estas **para reemplazar todas las listas utilizadas en el código**, de manera de hacerlo más eficiente y justifique por qué la nueva implementación funciona mejor.

- A)

```
class ListaDeEspera:
    def __init__(self):
        self.esperando = []

    def agregar_persona(self, persona):
        self.esperando.append(persona)

    def siguiente_persona(self):
        return self.esperando.pop(0)
```
- B)

```
class RegistroPuntosDeInteres:
    def __init__(self):
        self.puntos_de_interes = []

    def agregar_punto_de_interes(self, x, y):
        nuevo_punto = [x, y]
        existe = False
        for punto in self.puntos_de_interes:
            if nuevo_punto.x == punto.x and nuevo_punto.y == punto.y:
                existe = True
                break
        if not existe:
            self.puntos_de_interes.append(nuevo_punto)
```
- C)

```
class FormularioUsuarios:
    def __init__(self):
        self.usuarios = []

    def agregar_datos(self, rut, nombre, fecha_nacimiento, direccion):
        self.usuarios.append([rut, nombre, fecha_nacimiento, direccion])

    def buscar_dato(self, rut, valor_a_buscar):
        for usuario in self.usuarios:
            if usuario[0] == rut:
                if valor_a_buscar == "nombre":
                    return usuario[1]
                elif valor_a_buscar == "fecha_nacimiento":
                    return usuario[2]
                else:
                    return usuario[3]
```

Pregunta 2. [20 % nota final]

A partir de la siguiente descripción, debes construir un modelo orientado a objetos que permita capturar los requisitos que indica la descripción:

Un estudio de videojuegos te ha contratado para diseñar el sistema de clases de su nuevo juego de rol. Antes de escribir una sola línea de código, el equipo necesita un modelo claro que refleje cómo se relacionan los distintos elementos del juego.

En este juego existen personajes, que pueden ser de dos tipos: héroe o enemigo. Todo personaje tiene un nombre, puntos de vida y un nivel, y todos son capaces de atacar y recibir daño. Sin embargo, la forma en que atacan difiere: un héroe usa su nivel como daño base, mientras que un enemigo tiene un daño fijo determinado por su dificultad, que puede ser “fácil” (5 puntos de daño) o “difícil” (15 puntos de daño).

El nivel de un personaje está protegido: nunca puede bajar de 1, por lo que deberá modelarse con la validación correspondiente.

El héroe, además de lo anterior, carga un inventario de objetos y puede usarlos durante el combate. El enemigo, por su parte, puede intentar huir del combate, acción que puede o no resultar exitosa.

Los objetos que el héroe puede cargar tienen un nombre y un efecto descriptivo. Existen dos tipos concretos: las pociones, que además especifican cuántos puntos de vida restauran, y las armas, que especifican cuánto daño extra otorgan.

Todo esto ocurre dentro de una partida. La partida nace con un héroe asignado y no tiene sentido sin él: si el héroe desaparece, la partida también. Los enemigos, en cambio, aparecen y son derrotados a lo largo del juego de forma independiente. La partida expone dos acciones principales: iniciar y finalizar el juego.

Responde lo siguiente:

- A) Siguiendo el formato visto en el curso, construye el **diagrama de clases UML** que modele este sistema. Indica claramente los atributos con sus tipos, los métodos con sus parámetros, sus tipos y el tipo de la variable de retorno, las *properties*, y el tipo de relación entre cada par de clases (herencia, composición o agregación).
- B) A partir de tu diagrama, escribe el pseudocódigo pythonesco de las clases personaje, héroe, enemigo y partida. Solamente escribe el pseudocódigo de estas cuatro clases, independiente de si hay mas clases en tu diagrama. No es necesario implementar la lógica interna de los métodos, excepto en:
- El `__init__` de cada clase debe inicializar todos sus atributos y usar `super()` donde corresponda.
 - Cada *property* debe incluir su *getter* y *setter*.
 - El método `atacar()` debe estar implementado con la lógica correcta en héroe y enemigo.

Nota: Un pseudocódigo «pythonesco» es aquel que, si bien no requiere sintaxis perfecta, sí contiene los componentes estructurales clave (`class`, `def`, `self`, `super()`, decoradores `@property`) y las llamadas necesarias para que la lógica sea correcta.

Pregunta 3. [20 % nota final]

Esta pregunta consta de dos partes, las cuales no se encuentran relacionadas.

Parte A. [3 puntos]

Se te ha pedido simular usando pseudocódigo la inscripción de grupos en una plataforma la cual se simula en la instancia de clase `Plataforma`. Para esto deberás:

- Permitir el ingreso de un usuario a la plataforma mediante su número de alumno para luego poder tener acciones. Si el usuario ingresa un número de alumno inexistente, se le deberá mostrar nuevamente la opción de ingresar el número de alumno. Para evaluar si existe el número de alumno ingresado existe la función `Plataforma.existe_alumno(num_alumno)`, que retorna **True**, si existe el número de alumno; y **False**, en otro caso.
- Una vez que se tenga el número del alumno del usuario, deben ofrecer las siguientes **acciones sobre grupos**:
 1. Mostrar los miembros (usando el número de alumno de cada uno) del grupo al que pertenece el usuario. Deberás usar la función `Plataforma.grupo_usuario(num_alumno)`, que retorna los miembros del grupo del usuario separados por coma, y **None**, si el usuario no posee grupo. Si el usuario no pertenece a un grupo, se le indica explícitamente que no tiene grupo.
 2. Crear un grupo con solo el usuario. Puedes suponer que si el usuario escoge esta opción, este no posee grupo. Para crear el grupo deberás usar `Plataforma.crear_grupo(num_alumno)`
 3. Ser agregado al grupo de un compañero pidiéndole al usuario ingresar el número de alumno del otro alumno. Puedes suponer que el usuario ingresará un número de alumno de compañero válido y que su compañero poseerá grupo. Para esta funcionalidad, deberás usar la función: `Plataforma.moverse_a_grupo(num_alumno_mover, num_alumno_grupo)`
 4. Salir del menú y dejar de interactuar con la plataforma. Si un usuario no ingresa explícitamente alguna de las **acciones sobre grupos** listada, se considera que seleccionó la acción de salir.

Luego de completar una acción distinta a Salir, el programa debe permitir volver a escoger una **acción sobre grupos**.

La plataforma se emulará usando interacción por consola.

Parte B. [3 puntos]

Dada la siguiente función:

```
def es_sucesion_fibonacci(primer, segundo, tercero=None, *otros):
    if tercero is None:
        return True
    es_fibonacci = ((primer + segundo) == tercero)
    viene_fibonacci = es_sucesion_fibonacci(segundo, tercero, *otros)
    return es_fibonacci and viene_fibonacci
```

Deberás rellenar la tabla de la hoja de respuestas usando el orden en el que se llama esa función de forma recursiva, y usando el formato que se ejemplifica a continuación. El llamado inicial es:

`es_sucesion_fibonacci(0, 1, 1, 2, 2, 4, 6).`

Ejemplo de formato parte 3B

El siguiente es un ejemplo de llenado de tabla para una función recursiva usando el llamado:

`sumar_tres(2, 23, 3)`

```
def sumar_tres(a, b, c):
    if a == 25:
        return b + c
    ab = a + b
    return a + sumar_tres(ab, b, c)
```

#	a	b	c	ab	valor retornado
1	2	23	3	25	28
2	25	23	3		26

La columna “#” representa el número de llamada de la función.

Pregunta 4. [20 % nota final]

El Sudoku es un puzzle matemático donde el objetivo es rellenar una matriz de 9×9 , dividido en cuadrantes de 3×3 , con números del 1 al 9. La regla fundamental es que cada número del 1 al 9 debe aparecer solo una vez en cada fila, columna y cuadrante. Deberás crear un algoritmo que permita encontrar una solución para un tablero de Sudoku.

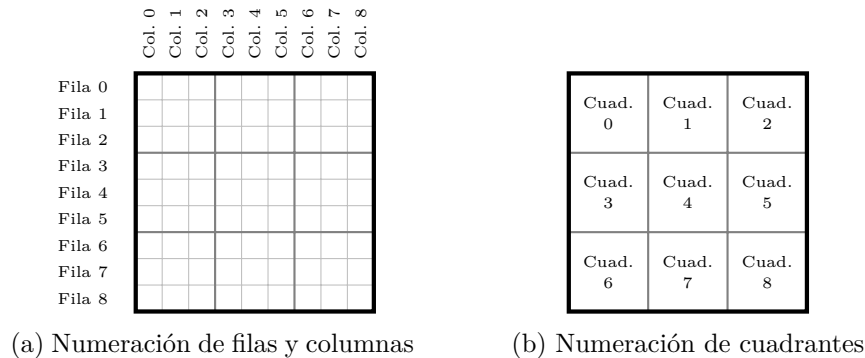


Figura 1: Numeración filas, columnas y cuadrantes

Para representar el estado de un tablero se utiliza una lista de listas de números enteros, donde se utilizan números del 1 al 9 para representar las celdas rellenas y 0 para representar las celdas vacías.

Puedes utilizar las siguientes funciones que **ya se encuentran implementadas**:

- `def encontrar_celda_vacia(tablero: list[list[int]]) -> tuple[int, int] | None:`
Recibe un tablero y retorna una tupla con las coordenadas de una celda que se encuentra vacía. Si no hay celdas vacías, entonces retorna **None**.
- `def encontrar_cuadrante(fila: int, columna: int) -> int:`
Recibe las coordenadas de una celda del tablero y retorna el índice del cuadrante que la contiene.
- `def comprobar_fila(tablero: list[list[int]], indice: int) -> bool:`
Recibe el estado de un tablero y el índice de una de sus filas (Figura 1a). Retorna un booleano que indica si dicha fila cumple las reglas del Sudoku, es decir, no presenta números del 1 al 9 repetidos.
- `def comprobar_columna(tablero: list[list[int]], indice: int) -> bool:`
Recibe el estado de un tablero y el índice de una de sus columnas (Figura 1a). Retorna un booleano que indica si dicha columna cumple las reglas del Sudoku, es decir, no presenta números del 1 al 9 repetidos.
- `def comprobar_cuadrante(tablero: list[list[int]], indice: int) -> bool:`
Recibe el estado de un tablero y el índice de uno de sus cuadrantes (Figura 1b). Retorna un booleano que indica si dicho cuadrante cumple las reglas del Sudoku, es decir, no presenta números del 1 al 9 repetidos.

A partir de la estructura de tableros mencionada anteriormente y las funciones entregadas, utiliza **pseudo-código** para crear una **función recursiva** que recibe el tablero como *input* y mediante el uso de **backtracking** retorne el tablero resuelto. Queda a tu criterio que retornar si no se encuentra solución.